

AuthProxy Endpoints Secured with OAuth 2.0 Bearer Tokens

Securing endpoints with bearer tokens adds an important extra layer of security by offloading the ownership of the secrets from the application (us) to the customer. **Also note that the AuthProxy *only* validates tokens and keys - it does not play any part in the flow to acquire the token!**

The AuthProxy supports three ways you can secure an endpoint with OAuth 2.0 Bearer Tokens: Pure authorization code grant, authorization code grant in combination with authkey and client credential grant (also called two-legged oauth) in combination with authkey and a connected service user.

All samples in this document is using C# and Azure AD, but any compliant provider can be used.

Contents

Authorization Code Grant (ACG)

[Configure](#)

[Test](#)

Authorization Code Grant in Combination with AuthKey (ACGK)

[Configure](#)

[Test](#)

Client Credential Grant in Combination with AuthKey (CCGA)

[Configure](#)

[Test](#)

Authorization Code Grant (ACG)

Here the client is represented by a real user that typically will be prompted for username and password as part of the authentication flow as described here:

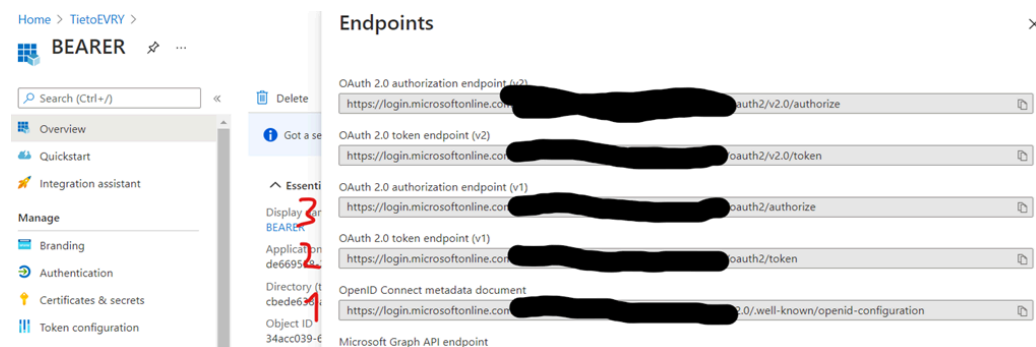
<https://docs.microsoft.com/en-us/azure/active-directory/develop/v2-oauth2-auth-code-flow>

Securing and endpoint in360 with ACG only makes sense when used on pure client access endpoints (endpoints designed to be called from client applications like Outlook add-in, Teams integration and as a React backends. The reason for this is that there is no way to separate access to different endpoints. If a users have access to endpoint A secured with ACG, they will also have access to endpoint B secured with ACG. What the user actually get access to in 360 is then controlled only by the internal 360 access control system.

Configure

In Azure AD (on your tenant), create an app registration and configure it to your need.

Then use the information below to test the endpoint using a tool like for instance Postman.



Configure the endpoint in the 360 administrator.

Test

Use the information above in Postman to test the endpoint.

This will prompt you with a dialog requiring your credentials before issuing the call against the endpoint.

Authorization Code Grant in Combination with AuthKey (ACGK)

This adds an extra dimension to the authentication by requiring a secret that is managed by the 360 product (responsible). The resources is required to have a "technical administrator role" in 360, and is typically different from the resources managing the token configuration. The authkey is also adding "high level" authorization by restricting access to specific functionality in 360 by the use of endpoint paths.

Configure

Configure the endpoint in the 360 administrator (with the additional authkey).

Add new endpoint

Third party application *

Integration point *

Authentication type *

Metadata URI *

User claims *

Use existing auth key

Valid from

To date

Client id

Save

Cancel

Test

Test with the same configuration as for ACG, but add the authkey either as an header or on the query string.

GET http://loc...

[DELETED] GET h

[DELETED] GET h

GET http://loc...

GET http://loc...X

+

...

No Environment

Untitled Request

BUILD

GET

http://localhost:3001/biz/v2-pbr/info/whoami

Send

Save

Params

Authorization

Headers (12)

Body

Pre-request Script

Tests

Settings

Cookies

Code

☒	Content-Length	<calculated when request is sent>	
☒	Host	<calculated when request is sent>	
☒	User-Agent	PostmanRuntime/7.26.8	
☒	Accept	*/*	
☒	Accept-Encoding	gzip, deflate, br	
☒	Connection	keep-alive	
☒	Cookie	ApplicationGatewayAffinity=f8ed4c744b9f3cfa54...	
☒	AuthKey	XXXX	
	Key	Value	Description

Body

Cookies

Headers

Test Results

Status: 401 Unauthorized

Time: --

Size: 732 B

Save Response

Pretty

Raw

Preview

Visualize

Text

1

Client Credential Grant in Combination with AuthKey (CCGA)

Client credential grant can not be used by its own without introducing additional information to identify the different clients:

<https://docs.microsoft.com/en-us/azure/active-directory/develop/v2-oauth2-client-creds-grant-flow>

This is the option to use when the client is acting on behalf of the user or as a separate application (daemon). The main "problem" here is that there is no user connected to the token. This means that you have to use some "property" on the token as identification, or as we do, connect the token explicitly to a 360 user. The coupling of the token and user is done on a given endpoint with a given authkey.

Configure

Configure mostly as ACG, but consider using customer certificates instead of Azure secrets.

Configure in 360

Add new endpoint

Third party application *

Integration point *

Authentication type *

Combined daemon

Metadata URI *

https://...

Use existing auth key

No

Valid from

To date

Service user

Create new service user (Recommended)

Permissions *

Client id

Save

Cancel

Test

Extract from a small c# sample application running as a demon.

https://tieto-si.visualstudio.com/360/_git/core?path=%2Fsamples%2Fauthproxy%2FOauth2o%2Fdaemon

```

1  using Microsoft.Identity.Client;
2  using Newtonsoft.Json.Linq;
3  using System;
4  using System.Collections.Generic;
5  using System.Linq;
6  using System.Net.Http;
7  using System.Security.Cryptography.X509Certificates; //Only import this if you are using certificate
8  using System.Threading.Tasks;
9
10 namespace daemon
11 {
12     class Program
13     {
14         static void Main(string[] args)
15         {
16             try
17             {
18                 RunAsync().GetAwaiter().GetResult();
19             }
20             catch (Exception ex)
21             {
22                 Console.ForegroundColor = ConsoleColor.Red;
23                 Console.WriteLine(ex.Message);
24                 Console.ResetColor();
25             }
26
27             Console.WriteLine("Press any key to exit");
28             Console.ReadKey();
29         }
30
31         private static async Task RunAsync()
32         {
33             AuthenticationConfig config = AuthenticationConfig.ReadFromJsonFile("appsettings.json");
34
35             // You can run this sample using ClientSecret or Certificate. The code will differ only when instantiating the
36             IConfidentialClientApplication
37             bool isUsingClientSecret = AppUsesClientSecret(config);
38
39             // Even if this is a console application here, a daemon application is a confidential client application
40             IConfidentialClientApplication app;
41
42             if (isUsingClientSecret)
43             {
44                 app = ConfidentialClientApplicationBuilder.Create(config.ClientId)
45                     .WithClientSecret(config.ClientSecret)
46                     .WithAuthority(new Uri(config.Authority))
47                     .Build();
48             }
49             else
50             {
51                 X509Certificate2 certificate = ReadCertificate(config.CertificateName);
52                 app = ConfidentialClientApplicationBuilder.Create(config.ClientId)
53                     .WithCertificate(certificate)
54                     .WithAuthority(new Uri(config.Authority))
55                     .Build();
56             }
57
58             // With client credentials flows the scopes is ALWAYS of the shape "resource/.default", as the
59             // application permissions need to be set statically (in the portal or by PowerShell), and then granted by
60             // a tenant administrator

```

```

61     string[] scopes = new string[] { config.Scope };
62
63     AuthenticationResult result = null;
64     try
65     {
66         result = await app.AcquireTokenForClient(scopes)
67             .ExecuteAsync();
68         Console.ForegroundColor = ConsoleColor.Green;
69         Console.WriteLine("Token acquired \n");
70         Console.ResetColor();
71     }
72     catch (MsalServiceException ex) when (ex.Message.Contains("AADSTS70011"))
73     {
74         // Invalid scope. The scope has to be of the form "https://resourceurl/.default"
75         // Mitigation: change the scope to be as expected
76         Console.ForegroundColor = ConsoleColor.Red;
77         Console.WriteLine("Scope provided is not supported");
78         Console.ResetColor();
79     }
80
81     if (result != null)
82     {
83         var httpClient = new HttpClient();
84         var apiCaller = new ProtectedApiCallHelper(httpClient);
85         await apiCaller.CallWhoAmI(config.BaseAddress, result.AccessToken, config.AuthKey);
86     }
87 }
88
89 private static void Display(IEnumerable<JObject> result)
90 {
91     Console.WriteLine("Web Api result: \n");
92
93     foreach (var item in result)
94     {
95         foreach (JProperty child in item.Properties().Where(p => !p.Name.StartsWith("@")))
96         {
97             Console.WriteLine($"{child.Name} = {child.Value}");
98         }
99
100         Console.WriteLine("");
101     }
102 }
103
104 /// <summary>
105 /// Checks if the sample is configured for using ClientSecret or Certificate. This method is just for the sake of this sample.
106 /// You won't need this verification in your production application since you will be authenticating in AAD using one mechanism
107 only.
108 /// </summary>
109 /// <param name="config">Configuration from appsettings.json</param>
110 /// <returns></returns>
111 private static bool AppUsesClientSecret(AuthenticationConfig config)
112 {
113     string clientSecretPlaceholderValue = "[Enter here a client secret for your application]";
114     string certificatePlaceholderValue = "[Or instead of client secret: Enter here the name of a certificate (from the user cert store) as registered with your application]";
115
116     if (!String.IsNullOrEmpty(config.ClientSecret) && config.ClientSecret != clientSecretPlaceholderValue)
117     {
118         return true;
119     }
120
121     else if (!String.IsNullOrEmpty(config.CertificateName) && config.CertificateName != certificatePlaceholderValue)
122     {
123         return false;
124     }
125
126     else
127     {
128         throw new Exception("You must choose between using client secret or certificate. Please update appsettings.json file.");
129     }
130 }
131
132 private static X509Certificate2 ReadCertificate(string certificateName)
133 {
134     if (string.IsNullOrEmpty(certificateName))
135     {
136         throw new ArgumentException("certificateName should not be empty. Please set the CertificateName setting in the appsettings.json", "certificateName");
137     }
138     X509Certificate2 cert = null;
139
140     using (X509Store store = new X509Store(StoreName.My, StoreLocation.CurrentUser))
141     {
142         store.Open(OpenFlags.ReadOnly);
143         X509Certificate2Collection certCollection = store.Certificates;
144
145         // Find unexpired certificates.
146         X509Certificate2Collection currentCerts = certCollection.Find(X509FindType.FindByTimeValid, DateTime.Now, false);
147
148         // From the collection of unexpired certificates, find the ones with the correct name.
149         X509Certificate2Collection signingCert = currentCerts.Find(X509FindType.FindBySubjectDistinguishedName, certificateName, false);
150
151         // Return the first certificate in the collection, has the right name and is current.
152         cert = signingCert.OfType<X509Certificate2>().OrderByDescending(c => c.NotBefore).FirstOrDefault();
153     }
154     return cert;
155 }
156
157 }
158
159 using System;
160 using System.Net.Http;
161 using System.Net.Http.Headers;
162 using System.Threading.Tasks;

```

```

162
163 namespace daemon
164 {
165     /// <summary>
166     /// Helper class to call a protected API and process its result
167     /// </summary>
168     public class ProtectedApiCallHelper
169     {
170         public ProtectedApiCallHelper(HttpClient httpClient)
171         {
172             HttpClient = httpClient;
173         }
174
175         protected HttpClient HttpClient { get; private set; }
176
177
178         public async Task CallWhoAmI(string baseAddress, string accessToken, string authKey)
179         {
180             if (!string.IsNullOrEmpty(accessToken))
181             {
182                 var defaultRequestHeaders = HttpClient.DefaultRequestHeaders;
183                 defaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", accessToken);
184                 defaultRequestHeaders.Add("authkey", authKey);
185
186                 HttpResponseMessage response = await HttpClient.GetAsync($"{baseAddress}/biz/v2-pbr/info/whoami");
187                 if (response.IsSuccessStatusCode)
188                 {
189                     string res = await response.Content.ReadAsStringAsync();
190                     Console.ForegroundColor = ConsoleColor.Green;
191                     Console.WriteLine($"You are '{res}'");
192                     Console.ForegroundColor = ConsoleColor.Gray;
193                 }
194                 else
195                 {
196                     Console.ForegroundColor = ConsoleColor.Red;
197                     Console.WriteLine($"Failed to call the web API: {response.StatusCode}");
198                     string content = await response.Content.ReadAsStringAsync();
199
200                     // Note that if you got response.Code == 403 and response.content.code == "Authorization_RequestDenied"
201                     // this is because the tenant admin has not granted consent for the application to call the Web API
202                     Console.WriteLine($"Content: {content}");
203                 }
204                 Console.ResetColor();
205             }
206         }
207     }
208 }
209
210 using Microsoft.Extensions.Configuration;
211 using System;
212 using System.Globalization;
213 using System.IO;
214
215 namespace daemon
216 {
217     /// <summary>
218     /// Description of the configuration of an AzureAD public client application (desktop/mobile application). This should
219     /// match the application registration done in the Azure portal
220     /// </summary>
221     public class AuthenticationConfig
222     {
223         /// <summary>
224         /// instance of Azure AD, for example public Azure or a Sovereign cloud (Azure China, Germany, US government, etc ...)
225         /// </summary>
226         public string Instance { get; set; } = "https://login.microsoftonline.com/{0}";
227
228         /// <summary>
229         /// The Tenant is:
230         /// - either the tenant ID of the Azure AD tenant in which this application is registered (a guid)
231         /// or a domain name associated with the tenant
232         /// - or 'organizations' (for a multi-tenant application)
233         /// </summary>
234         public string Tenant { get; set; }
235
236         /// <summary>
237         /// Guid used by the application to uniquely identify itself to Azure AD
238         /// </summary>
239         public string ClientId { get; set; }
240
241         /// <summary>
242         /// URL of the authority
243         /// </summary>
244         public string Authority
245         {
246             get
247             {
248                 return String.Format(CultureInfo.InvariantCulture, Instance, Tenant);
249             }
250         }
251
252         /// <summary>
253         /// Client secret (application password)
254         /// </summary>
255         /// <remarks>Daemon applications can authenticate with AAD through two mechanisms: ClientSecret
256         /// (which is a kind of application password: this property)
257         /// or a certificate previously shared with AzureAD during the application registration
258         /// (and identified by the CertificateName property belows)
259         /// </remarks>
260         public string ClientSecret { get; set; }
261
262         /// <summary>
263         /// Name of a certificate in the user certificate store
264         /// </summary>
265         /// <remarks>Daemon applications can authenticate with AAD through two mechanisms: ClientSecret
266         /// (which is a kind of application password: the property above)

```

```

267     /// or a certificate previously shared with AzureAD during the application registration
268     /// (and identified by this CertificateName property)
269     /// <remarks>
270     public string CertificateName { get; set; }
271
272     /// <summary>
273     /// Web Api base URL
274     /// </summary>
275     public string BaseAddress { get; set; }
276
277     /// <summary>
278     /// Web Api scope. With client credentials flows, the scopes is ALWAYS of the shape "resource/.default"
279     /// </summary>
280     public string Scope { get; set; }
281
282     /// <summary>
283     /// Optional authkey
284     /// </summary>
285     public string AuthKey { get; set; }
286
287     public static AuthenticationConfig ReadFromJsonFile(string path)
288     {
289         IConfigurationRoot Configuration;
290
291         var builder = new ConfigurationBuilder()
292             .SetBasePath(Directory.GetCurrentDirectory())
293             .AddJsonFile(path);
294
295         Configuration = builder.Build();
296         return Configuration.Get<AuthenticationConfig>();
297     }
298 }
299

```

Retrieved from "https://wiki.software-innovation.com/index.php?title=AuthProxy_Endpoints_Secured_with_OAuth_2.0_Bearer_Tokens&oldid=32632"

This page was last edited on 21 December 2021, at 15:22.